

Off-Policy learning and comparison with SARSA

Instructor

Dr. Mayank Baranwal

Senior Scientist, TCS Research



Off-Policy Control

Why use Off-Policy Learning?

- On-policy learning algorithms evaluate and improve the same policy that is used to sample actions.

Off-Policy Control

Learning characteristics

Optimality:

- Off-policy methods train an optimal policy directly, thus have **better chances** of arriving at global optimal solution

Online performance:

- On-policy methods may **perform better** in some cases since the latest version of learned policy is used for action selection

Sample efficiency:

- On-policy methods are relatively **less sample-efficient** since they rely on single policy

Off-Policy Control

Learning characteristics

Flexibility:

- Off-policy methods are **more flexible** in terms of learning from diverse data sources

Convergence:

- On-policy methods, such as SARSA, provide better convergence guarantees under milder assumptions

Complexity:

- On-policy methods are easy-to-tune since a single policy needs to be stored

Importance Sampling

- Evaluate target policy $\pi(a|s)$ while following behavior policy $\mu(a|s)$

$$\{S_1, A_1, R_2, \dots, S_T\} \sim \mu$$

- Importance sampling allows to estimate expectation of a different distribution
- Recall that we want to evaluate the target policy π using returns generated from μ

Importance Sampling

- If we have two distributions P and Q, the expectations are related as:

$$\begin{aligned}\mathbb{E}_{X \sim P}[f(X)] &= \sum P(X) f(X) \\ &= \sum Q(X) \frac{P(X)}{Q(X)} f(X) \\ &= \mathbb{E}_{X \sim Q} \left[\frac{P(X)}{Q(X)} f(X) \right]\end{aligned}$$

Off-Policy TD using Importance Sampling

- On-policy TD learning:

$$v(S_t) \leftarrow v(S_t) + \alpha(R_{t+1} + \gamma v(S_{t+1}) - v(S_t))$$

- Off-policy TD learning:

$$v(S_t) \leftarrow v(S_t) + \alpha \left(\frac{\pi(A_t|S_t)}{\mu(A_t|S_t)} (R_{t+1} + \gamma v(S_{t+1})) - v(S_t) \right)$$

Q-learning – avoids Importance Sampling

- Importance sampling is not needed for learning action-values $Q(s, a)$
- Because,
 - Next action can be chosen using behavior policy $A_{t+1} \sim \mu(\cdot | S_t)$
 - Alternative action can follow $A' \sim \pi(\cdot | S_t)$ and update $Q(S_t, A_t)$

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma Q(S_{t+1}, A') - Q(S_t, A_t))$$

Q-learning – avoids Importance Sampling

- Target policy π is greedy w.r.t. $Q(s, a)$

$$\pi(S_{t+1}) = \arg \max_{a'} Q(S_{t+1}, a')$$

- Behavior policy μ is ϵ -greedy w.r.t. $Q(s, a)$
- Thus, Q-learning simplifies to:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left(R_{t+1} + \gamma \max_{A'} Q(S_{t+1}, A') - Q(S_t, A_t) \right)$$

Q-learning Algorithm for Off-Policy Control

```
Initialize  $Q(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$ , arbitrarily, and  $Q(\text{terminal-state}, \cdot) = 0$   
Repeat (for each episode):  
  Initialize  $S$   
  Repeat (for each step of episode):  
    Choose  $A$  from  $S$  using policy derived from  $Q$  (e.g.,  $\epsilon$ -greedy)  
    Take action  $A$ , observe  $R, S'$   
     $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$   
     $S \leftarrow S'$   
  until  $S$  is terminal
```

Thank You
